```
data <- read.csv("C:/Users/SBS/Desktop/4DS1/Statistic/project/stress_sommeil_longitudinal.csv")

head(data)

str(data)

summary(data)

sum(is.na(data))

sum(duplicated(data))

missing_summary <- sapply(data, function(x) sum(is.na(x)))
missing_summary
missing_percentage <- sapply(data, function(x) mean(is.na(x)) * 100)
missing_percentage

boxplot(data$stress_T1, main="Boxplot of Stress_T1", ylab="Stress_T1")
boxplot(data$anxiete_T2, main="Boxplot of Anxiety_T2", ylab="Anxiety_T2")
boxplot(data$sommeil_T3, main="Boxplot of Sleep_T3", ylab="Sleep_T3")

# Function to detect outliers using IQR method
detect_outliers <- function(x) {
  Q1 <- quantile(x, 0.25, na.rm = TRUE)
  Q3 <- quantile(x, 0.75, na.rm = TRUE)
  IQR <- Q3 - Q1
  lower <- Q1 - 1.5 * IQR
  upper <- Q3 + 1.5 * IQR
  return(x < lower | x > upper)
}

# Detect outliers for numerical variables
numerical_vars <- c("age", "travail_heures", "cafeine_tasses", "sport_jours",
                    "ecran_soir_heures", "stress_T1", "anxiete_T2", "sommeil_T3")

outliers_summary <- list()
for(var in numerical_vars) {
  outliers <- detect_outliers(data[[var]])
  outliers_summary[[var]] <- sum(outliers, na.rm = TRUE)
}

print(outliers_summary)

# Visualize outliers
numeric_data <- data[, sapply(data, is.numeric)]

# Create boxplot for all numeric columns
boxplot(numeric_data,
        main="Boxplots of Numeric Features",
        col="lightblue",
        las=2, # rotate axis labels for readability
        ylab="Values")

#feature:Gender
table(data$sexe, useNA = "always")


#print('**************DATA PREPARATION****************')

# Clean the gender column


unique(data$sexe)
# Replace "Fem " with "F"
data$sexe[data$sexe == "Fem "] <- "F"

# Replace "hom" with "H"
data$sexe[data$sexe == "hom"] <- "H"

# Replace "??" with NA
data$sexe[data$sexe == "??"] <- NA
```

```
#travail_heures ******************************

numeric_cols <- sapply(data, is.numeric)

for(col in names(data)[numeric_cols]){
  Q1 <- quantile(data[[col]], 0.25, na.rm=TRUE)
  Q3 <- quantile(data[[col]], 0.75, na.rm=TRUE)
  IQR_value <- Q3 - Q1
  lower <- Q1 - 1.5 * IQR_value
  upper <- Q3 + 1.5 * IQR_value

  # Extract outlier values
  outliers <- data[[col]][data[[col]] < lower | data[[col]] > upper]

  # Print results
  if(length(outliers) == 0){
    cat("Column", col, "has no outliers.\n")
  } else {
    cat("Column", col, "has", length(outliers), "outlier(s). The values are:\n")
    for(val in outliers){
      cat(" - Outlier value for", col, ":", val, "\n")
    }
  }
  cat("\n--------------------------\n")
}

# Remove outliers and missing values, then calculate the range
valid_stress <- data$stress_T1[!data$stress_T1 %in% c(-5, 25, 50) & !is.na(data$stress_T1)]

# Calculate and display the proven interval
cat("PROVEN STRESS INTERVAL (excluding outliers):\n")
cat("Minimum stress score:", min(valid_stress), "\n")
cat("Maximum stress score:", max(valid_stress), "\n")
cat("Range:", range(valid_stress), "\n")
cat("Number of valid observations:", length(valid_stress), "\n\n")

# Show the actual min and max values in the data
cat("The minimum value", min(valid_stress), "appears in rows:", which(data$stress_T1 == min(valid
cat("The maximum value", max(valid_stress), "appears in row:", which(data$stress_T1 == max(valid_

# Optional: Show first few valid values to demonstrate
cat("\nFirst 10 valid stress values:\n")
head(valid_stress, 10)




# Replace the specific outliers 120 and 200 with NA
data$travail_heures[data$travail_heures %in% c(120, 200)] <- NA

# Check if it worked
data$travail_heures[data$travail_heures %in% c(120, 200)]  # should return nothing
sum(is.na(data$travail_heures))  # counts all NAs including these new ones


# Replace the specific outliers 50 ,25 and -5 with NA
data$stress_T1[data$stress_T1 %in% c(50 ,25 , -5)] <- NA

# Check if it worked
data$stress_T1[data$stress_T1 %in% c(50 ,25 , -5)]  # should return nothing
sum(is.na(data$stress_T1))  # counts all NAs including these new ones




# Replace the specific outliers -2  with NA
data$sommeil_T3[data$sommeil_T3 %in% c( -2)] <- NA

# Check if it worked
data$sommeil_T3[data$sommeil_T3 %in% c( -2)]  # should return nothing
sum(is.na(data$sommeil_T3))  # counts all NAs including these new ones
```

```
summary(data)

#****************Missing Values***********************

#print('**************DATA Imputation****************')
imputed_data <- data
install.packages("VIM")
library(VIM)
# Perform KNN imputation on selected variables with missing data
# Here we specify k = 5 neighbors, you can change k to 3 or other
# 'variable' specifies which columns to impute, e.g., "age", "travail_heures", "stress_T1", "anxi
imputed_data <- kNN(imputed_data, variable = c("age", "travail_heures", "stress_T1", "anxiete_T2"



imputed_data <- kNN(
  data,
  variable = c("age", "travail_heures", "stress_T1", "anxiete_T2", "sommeil_T3"),
  k = 5,
  imp_var = FALSE
)

summary(imputed_data)
# View imputed data
head(imputed_data)


#******************Metrics***********************
# 1. BASIC MISSING VALUES COMPARISON
cat("=== MISSING VALUES COMPARISON ===\n")

# Identify numeric columns
numeric_vars <- names(data)[sapply(data, is.numeric)]

# Compare missing values before and after imputation
missing_comparison <- data.frame(
  Variable = numeric_vars,
  Missing_Before = sapply(data[numeric_vars], function(x) sum(is.na(x))),
  Missing_After  = sapply(imputed_data[numeric_vars], function(x) sum(is.na(x)))
)

# Print the result
print(missing_comparison)




# 2. DESCRIPTIVE STATISTICS COMPARISON
cat("\n=== DESCRIPTIVE STATISTICS COMPARISON ===\n")

desc_stats <- data.frame(
  Variable = character(),
  Mean_Before = numeric(),
  Mean_After = numeric(),
  SD_Before = numeric(),
  SD_After = numeric(),
  stringsAsFactors = FALSE
)

for(var in numeric_vars) {
  desc_stats <- rbind(desc_stats, data.frame(
    Variable = var,
    Mean_Before = round(mean(data[[var]], na.rm = TRUE), 2),       # original data
    Mean_After  = round(mean(imputed_data[[var]], na.rm = TRUE), 2), # imputed data
    SD_Before   = round(sd(data[[var]], na.rm = TRUE), 2),
    SD_After    = round(sd(imputed_data[[var]], na.rm = TRUE), 2)
  ))
}

print(desc_stats)
```

```
# 4. CORRELATION PRESERVATION
cat("\n=== CORRELATION PRESERVATION ===\n")

# Compute correlations
cor_before <- cor(data[numeric_vars], use = "complete.obs")
cor_after  <- cor(imputed_data[numeric_vars], use = "complete.obs")

# Focus on key relationships
key_pairs <- list(
  c("stress_T1", "sommeil_T3"),
  c("stress_T1", "anxiete_T2"),
  c("anxiete_T2", "sommeil_T3")
)

# Prepare comparison table
cor_comparison <- data.frame(
  Relationship = character(),
  Cor_Before = numeric(),
  Cor_After = numeric(),
  Difference = numeric(),
  stringsAsFactors = FALSE
)

# Fill comparison table
for(pair in key_pairs) {
  cor_bef <- cor_before[pair[1], pair[2]]
  cor_aft <- cor_after[pair[1], pair[2]]

  cor_comparison <- rbind(cor_comparison, data.frame(
    Relationship = paste(pair[1], "vs", pair[2]),
    Cor_Before = round(cor_bef, 3),
    Cor_After = round(cor_aft, 3),
    Difference = round(cor_aft - cor_bef, 3)
  ))
}

# Print results
print(cor_comparison)


# 5. QUICK VISUAL CHECK
cat("\n=== CREATING QUICK VISUAL COMPARISON ===\n")

# Set up plotting area: 2 rows, 4 columns
par(mfrow = c(2, 4))

for(var in numeric_vars) {
  # Histogram before imputation
  hist(data[[var]],
       main = paste("Before:", var),
       xlab = var,
       col = "red",
       breaks = 15)

  # Histogram after imputation
  hist(imputed_data[[var]],
       main = paste("After:", var),
       xlab = var,
       col = "lightblue",
       breaks = 15)
}

# Reset plotting area
par(mfrow = c(1, 1))


# 6. FINAL SUMMARY
cat("\n=== IMPUTATION SUMMARY ===\n")

# Total missing values before and after
total_missing_before <- sum(is.na(data[numeric_vars]))
total_missing_after  <- sum(is.na(imputed_data[numeric_vars]))

# Imputation success rate
```

```
success_rate <- round((total_missing_before - total_missing_after) / total_missing_before * 100,

# Print summary
cat("Total missing values before:", total_missing_before, "\n")
cat("Total missing values after: ", total_missing_after, "\n")
cat("Imputation success rate: ", success_rate, "%\n")


#******************Data Transformation*****************

#******************SCaling*****************
scaling <- imputed_data
# Min-Max scaling function

min_max_scale <- function(x) {
  (x - min(x, na.rm = TRUE)) / (max(x, na.rm = TRUE) - min(x, na.rm = TRUE))
}

# Apply to all numeric columns in your dataframe (excluding categorical/factor variables e.g. 'se
numeric_vars <- sapply(scaling, is.numeric)
df_minmax <- scaling
df_minmax[numeric_vars] <- lapply(scaling[numeric_vars], min_max_scale)


head(df_minmax)


scaling <- impute_data_2
# Select the numeric variables you want to scale
numeric_vars <- c("age", "travail_heures", "cafeine_tasses", "sport_jours",
                  "ecran_soir_heures", "stress_T1", "anxiete_T2", "sommeil_T3")

# Create a scaled version of your imputed dataset
scaling <- impute_data_2  # start from imputed data

# Apply Z-score standardization
scaling[numeric_vars] <- scale(impute_data_2[numeric_vars])

# Check the result
summary(scaling)




#************************************************************



#******************Statistical Analysis*****************

# 0. Install / load required packages (if missing)
packages <- c("tseries", "car", "lmtest", "sandwich", "nortest")
install_if_missing <- function(pk){
  if (!requireNamespace(pk, quietly = TRUE)) install.packages(pk)
}
invisible(lapply(packages, install_if_missing))
library(tseries)    # jarque.bera.test
library(car)        # vif, etc.
library(lmtest)     # bptest
library(sandwich)   # robust SEs if needed
library(nortest)    # ad.test (Anderson-Darling) optional

# 1. Keep only quantitative variables (numeric)
quant_vars <- imputed_data[, sapply(imputed_data, is.numeric)]
cat("Numeric variables kept:\n"); print(names(quant_vars))


# 2. Normality tests (Shapiro-Wilk for n < 5000, Jarque-Bera otherwise)
normality_results <- data.frame(
  variable = character(),
  method = character(),
  p_value = numeric(),
  decision = character(),
  stringsAsFactors = FALSE
```

```
)

for (v in names(quant_vars)) {
  x <- quant_vars[[v]]
  x_clean <- x[!is.na(x)]
  if (length(x_clean) < 3) {
    pval <- NA
    method <- "too_small"
    decision <- "insufficient data"
  } else if (length(x_clean) < 5000) {
    test <- tryCatch(shapiro.test(x_clean), error = function(e) NULL)
    if (is.null(test)) {
      # fallback to Anderson-Darling
      test_ad <- tryCatch(ad.test(x_clean), error = function(e) NULL)
      pval <- if (!is.null(test_ad)) test_ad$p.value else NA
      method <- if (!is.null(test_ad)) "Anderson-Darling" else "shapiro_error"
    } else {
      pval <- test$p.value
      method <- "Shapiro-Wilk"
    }
    decision <- if (!is.na(pval) && pval > 0.05) "accept_H0_normal" else "reject_H0_not_normal"
  } else {
    jb <- tryCatch(jarque.bera.test(x_clean), error = function(e) NULL)
    pval <- if (!is.null(jb)) jb$p.value else NA
    method <- "Jarque-Bera"
    decision <- if (!is.na(pval) && pval > 0.05) "accept_H0_normal" else "reject_H0_not_normal"
  }
  normality_results <- rbind(normality_results, data.frame(
    variable = v, method = method, p_value = pval, decision = decision, stringsAsFactors = FALSE
  ))
}

cat("\n=== Normality test results ===\n")
print(normality_results)




 ##############################################################
# NORMALITY TESTS ON ORIGINAL DATA (before imputation)
##############################################################

cat("\n=== NORMALITY TESTS ON ORIGINAL DATA (NO IMPUTATION) ===\n")

# 1. Detect numeric variables in ORIGINAL dataset
numeric_vars_original <- names(data)[sapply(data, is.numeric)]
cat("Numeric variables detected in original data:\n")
print(numeric_vars_original)

# 2. Extract only numeric subset
original_numeric_data <- data[, numeric_vars_original]

# 3. Create empty result table
normality_original <- data.frame(
  variable = character(),
  method = character(),
  p_value = numeric(),
  decision = character(),
  stringsAsFactors = FALSE
)

# 4. Loop through numeric variables
for (v in names(original_numeric_data)) {

  x <- original_numeric_data[[v]]
  x_clean <- x[!is.na(x)]  # remove NAs first

  # Not enough values for test
  if (length(x_clean) < 3) {
    normality_original <- rbind(normality_original, data.frame(
      variable = v,
      method = "insufficient_data",
      p_value = NA,
      decision = "cannot_test",
```

```
      stringsAsFactors = FALSE
    ))
    next
  }

  # Choose test based on sample size
  if (length(x_clean) < 5000) {
    # Use Shapiro-Wilk
    test <- shapiro.test(x_clean)
    pval <- test$p.value
    method_used <- "Shapiro-Wilk"
  } else {
    # Use Jarque-Bera for large samples
    library(tseries)
    test <- jarque.bera.test(x_clean)
    pval <- test$p.value
    method_used <- "Jarque-Bera"
  }

  # Decision rule
  decision <- ifelse(pval > 0.05,
                     "accept_H0_normal",
                     "reject_H0_not_normal")

  # Add to table
  normality_original <- rbind(normality_original, data.frame(
    variable = v,
    method = method_used,
    p_value = pval,
    decision = decision,
    stringsAsFactors = FALSE
  ))
}

# 5. Display results
cat("\n===== NORMALITY RESULTS (ORIGINAL DATA) =====\n")
print(normality_original)



#****************** Non-Parametric Tests (Enhanced) ****************

# Install and load BSDA for Sign Test
if(!require(BSDA)) install.packages("BSDA")
library(BSDA)  # for SIGN.test

# Helper function to print interpretation
interpret_pval <- function(pval, alpha = 0.05, description = "Test") {
  if(is.na(pval)){
    cat(description, ": insufficient data or test failed\n")
  } else if(pval < alpha){
    cat(description, ": significant result (p <", alpha, ") => reject H0\n\n")
  } else {
    cat(description, ": not significant (p >=", alpha, ") => fail to reject H0\n\n")
  }
}

# 1. Mann-Whitney U Test: Compare two independent groups
# Scenario: Check if stress differs by gender
cat("=== Mann-Whitney U Test: stress_T1 by gender ===\n")

gender_var <- intersect(
  names(data),
  c("gender", "Gender", "sex", "Sex", "sexe")
)

if(length(gender_var) == 1){

  data[[gender_var]] <- as.factor(data[[gender_var]])

  mw_test <- wilcox.test(
    stress_T1 ~ data[[gender_var]],
    data = data,
    exact = FALSE
```

```
  )

  print(mw_test)
  interpret_pval(mw_test$p.value,
                 description = paste("Mann-Whitney U Test (stress ~", gender_var, ")"))

  if(require(ggplot2)){
    ggplot(data, aes(x = .data[[gender_var]], y = stress_T1)) +
      geom_boxplot(fill = "skyblue") +
      labs(title = "Stress by Gender (Mann-Whitney U Test)",
           x = gender_var, y = "Stress_T1")
  }

} else {
  cat("❌ Mann-Whitney test not run\n")
  cat("Reason: gender variable not found or ambiguous\n")
  print(names(data))
}

# 2⃣ Kruskal-Wallis Test: Compare more than 2 groups
# Scenario: Check if stress differs by sleep category
if("sleep_category" %in% names(data)){
  cat("=== Kruskal-Wallis Test: stress_T1 by sleep_category ===\n")
  kw_test <- kruskal.test(stress_T1 ~ sleep_category, data = data)
  print(kw_test)
  interpret_pval(kw_test$p.value, description = "Kruskal-Wallis Test (stress ~ sleep_category)")

  # Post-hoc pairwise Wilcoxon tests if KW is significant
  if(kw_test$p.value < 0.05){
    pairwise_res <- pairwise.wilcox.test(data$stress_T1, data$sleep_category, p.adjust.method = "
    cat("=== Pairwise Wilcoxon Test (post-hoc) ===\n")
    print(pairwise_res)
  }

  # Optional visualization
  if(require(ggplot2)){
    ggplot(data, aes(x = sleep_category, y = stress_T1)) +
      geom_boxplot(fill = "lightgreen") +
      labs(title = "Stress by Sleep Category (Kruskal-Wallis)", y = "Stress_T1", x = "Sleep Categ
  }
}

# 3⃣ Wilcoxon Signed-Rank Test: Paired comparison
# Scenario: Compare stress before vs after intervention/time
if(all(c("stress_T1", "stress_T2") %in% names(data))){
  cat("=== Wilcoxon Signed-Rank Test: stress_T1 vs stress_T2 ===\n")
  wilcox_paired <- wilcox.test(data$stress_T1, data$stress_T2, paired = TRUE)
  print(wilcox_paired)
  interpret_pval(wilcox_paired$p.value, description = "Wilcoxon Signed-Rank Test (paired stress)"
}

# 4⃣ Sign Test: Compare median to reference
# Scenario: Is the median stress different from 6?
if("stress_T1" %in% names(data)){
  cat("=== Sign Test: median stress_T1 vs 6 ===\n")
  sign_test <- SIGN.test(data$stress_T1, md = 6)
  print(sign_test)

  # Interpretation
  if(sign_test$p.value < 0.05){
    cat("Median stress_T1 significantly different from 6 => reject H0\n\n")
  } else {
    cat("Median stress_T1 not significantly different from 6 => fail to reject H0\n\n")
  }
}

# 5⃣ Spearman Correlation: Association between numeric variables
# Scenario: Check if stress is related to sleep
if(all(c("stress_T1", "sommeil_T3") %in% names(data))){
  cat("=== Spearman Correlation: stress_T1 vs sommeil_T3 ===\n")
  spearman_cor <- cor.test(data$stress_T1, data$sommeil_T3, method = "spearman")
  print(spearman_cor)

  # Interpretation
```

```
  cat("Interpretation: rho =", round(spearman_cor$estimate, 2), "\n")
  if(spearman_cor$p.value < 0.05){
    cat("Significant correlation => stress and sleep are associated\n\n")
  } else {
    cat("Not significant => stress and sleep not strongly associated\n\n")
  }

  # Optional scatter plot
  if(require(ggplot2)){
    ggplot(data, aes(x = sommeil_T3, y = stress_T1)) +
      geom_point(color = "orange") +
      geom_smooth(method = "loess") +
      labs(title = "Spearman Correlation: Stress vs Sleep", x = "Sleep_T3", y = "Stress_T1")
  }
}

#================ Key Insights =================#
cat("=== KEY INSIGHTS ===\n")
cat("- Non-normal variables confirmed: using non-parametric tests is appropriate.\n")
cat("- Significant Mann-Whitney/Kruskal-Wallis tests indicate important explanatory variables.\n"
cat("- Wilcoxon paired tests highlight changes over time.\n")
cat("- Sign test gives median comparison to reference (useful for risk thresholds).\n")
cat("- Spearman correlation identifies monotonic associations, guiding variable selection.\n")
```